

Some RDD transformations allow us to modify the parameters in the lambda expression rather than returning a new object. For example, in the sequence function of the aggregation function for `aggregateByKey` and `aggregate`, we can modify the original accumulator argument and define the combine function in such a way that the combination is created by modifying the first of the two accumulators. A common and effective paradigm for complicated aggregations is to define a Scala class with sequence and combine operations that return the existing object using the `this.type` annotations.

For example, suppose that we wanted to do some custom aggregation that is not already defined in Spark. Let's say that we have an RDD of key/value pairs where the keys are the panda's instructors and the values are the pupils' panda report cards. For each instructor we want to know the length of the longest word used, the average number of words used per report card, and the number of instances of the word "happy." One valid, easy-to-read approach would be to use the `aggregateByKey` function, which takes three arguments: a zero value that represents an empty accumulator, a sequence function that takes the accumulator and a value and adds the value to the accumulator, and a combine operator that defines how the accumulators should be combined. In this instance we could define our accumulator to be an object with four fields: the total count of all the words, the total number of reports, the longest word seen so far, and the total number of mentions of the word "happy."

For clarity we can define this as its own object with methods for sequence and combine. We have named this object `MetricsCalculator`, and it might be coded as shown in [Example 5-2](#).

Example 5-2. Custom aggregation object

```

class MetricsCalculator(
  val totalWords : Int,
  val longestWord: Int,
  val happyMentions : Int,
  val numberReportCards: Int) extends Serializable {

  def sequenceOp(reportCardContent : String) : MetricsCalculator =
  {
    val words = reportCardContent.split(" ")
    val tw = words.length
    val lw = words.map( w => w.length).max
    val hM = words.count(w => w.toLowerCase.equals("happy"))

    new MetricsCalculator(
      tw + totalWords,
      Math.max(longestWord, lw),
      hM + happyMentions,
      numberReportCards + 1)
  }

  def compOp(other : MetricsCalculator) : MetricsCalculator = {
    new MetricsCalculator(
      this.totalWords + other.totalWords,
      Math.max(this.longestWord, other.longestWord),
      this.happyMentions + other.happyMentions,
      this.numberReportCards + other.numberReportCards)
  }

  def toReportCardMetrics =
    ReportCardMetrics(
      longestWord,
      happyMentions,
      totalWords.toDouble/numberReportCards)
}

```

We could then use this object in the arguments to our aggregation function, as shown in [Example 5-4](#), in a routine that maps the RDD of instructors, and report text to a case class with the three metrics we care about in [Example 5-3](#).

Example 5-3. Case class for aggregations

```

case class ReportCardMetrics(
  longestWord : Int,
  happyMentions : Int,
  averageWords : Double)

```



Example 5-4. Aggregation example without object reuse

```
/**
 * Given an RDD of (PandaInstructor, ReportCardText) aggregate
 by instructor
 * to an RDD of distinct keys of (PandaInstructor,
 ReportCardStatistics)
 * where ReportCardMetrics is a case class with
 *
 * longestWord -> The longest word in all of the reports written
 by this instructor
 * happyMentions -> The number of times this instructor mentioned
 the word happy
 * averageWords -> The average number of words per report card
 for this instructor
 */
def calculateReportCardStatistics(rdd : RDD[(String, String)])
  : RDD[(String, ReportCardMetrics)] = {
  rdd.aggregateByKey(new MetricsCalculator(totalWords = 0,
    longestWord = 0, happyMentions = 0, numberReportCards = 0))(
    seqOp = ((reportCardMetrics, reportCardText) =>
      reportCardMetrics.sequenceOp(reportCardText)),
    combOp = (x, y) => x.compOp(y))
    .mapValues(_.toReportCardMetrics)
}
```



This method is superior to using a two `map` and one `reduceByKey` method. The `aggregate` function combines each partition locally, then does a shuffle to perform the cross-partition reduction. However, it has the disadvantage of creating a new instance of our custom object for each record in the dataset and for each combine step. A very simple way to reduce the cost of object creation would be to modify our `MetricsCalculator` to use Scala's `this.type` design paradigm so that the sequence operation modifies the original accumulator and the combine operation modifies the first accumulator rather than returning a new one, as shown in [Example 5-5](#).

Example 5-5. Aggregation example with object reuse

```
class MetricsCalculatorReuseObjects(
```

```

var totalWords : Int,
var longestWord: Int,
var happyMentions : Int,
var numberReportCards: Int) extends Serializable {

  def sequenceOp(reportCardContent : String) : this.type = {
    val words = reportCardContent.split(" ")
    totalWords += words.length
    longestWord = Math.max(longestWord, words.map( w =>
w.length).max)
    happyMentions += words.count(w =>
w.toLowerCase.equals("happy"))
    numberReportCards +=1
    this
  }

  def compOp(other : MetricsCalculatorReuseObjects) : this.type =
{
  totalWords += other.totalWords
  longestWord = Math.max(this.longestWord, other.longestWord)
  happyMentions += other.happyMentions
  numberReportCards += other.numberReportCards
  this
}

  def toReportCardMetrics =
    ReportCardMetrics(
      longestWord,
      happyMentions,
      totalWords.toDouble/numberReportCards)
}

```

Our aggregation routine will remain the same.

NOTE

It should be obvious that the Scala code within the sequence operator is slower than it needs to be. Rather than performing three different functional calls on the `words` array we ought to go through the string as a string buffer, counting the words, keeping track of the longest word, and counting the occurrence of the word “happy” (or at least use a while loop to parse the `words` array rather than three recursive calls). We have left this solution since we think it is easier to read and the primary intention of the example is to show how to optimize the `aggregateByKey` Spark routine.

Reduce (which calls `aggregate`) and the fold operations (`foldLeft`, `fold`, `foldRight`) can also benefit from object reuse. However, these aggregation functions are unique. It is best to avoid mutable data structures in Spark code (and Scala code in general) because they can lead to serialization errors and may have inaccurate results. For many other RDD functions, particularly narrow transformations, modifying the first value of the argument is not safe because the transformations may be chained together with lazy evaluation and may be evaluated multiple times. For example, if you have an RDD of mutable objects, modifying the arrays with a `map` function may lead to inaccurate results since the objects may be reused more times than you expect—especially if the RDD is recomputed.

Using Smaller Data Structures

Spark can be a memory hog. An important way to optimize Spark jobs for both time and space is to stick to primitive types rather than custom classes. Although it may make code less readable, using arrays rather than case classes or tuples can reduce GC overhead. Scala arrays, which are exactly Java arrays under the hood, are the most memory-efficient of the Scala collection types. Scala tuples are objects, so in some instances it might be better to use a two- or three-element array rather than a tuple for expensive operations. The Scala collection types in general incur a higher GC overhead than arrays.

Notice that our `ReportCardMetrics` object is just a wrapper for a few numeric values. Although it is less readable and less object-oriented, it is more space-efficient to use a four-element array of integers. We can maintain the same readable code paradigm by using a Scala `object` instead of a `class` and defining the sequence, and combine operations as

functions on strings and arrays as shown in [Example 5-6](#).

Example 5-6. Using an array as the aggregation object

```
object MetricsCalculator_Arrays extends Serializable {
  val totalWordIndex = 0
  val longestWordIndex = 1
  val happyMentionsIndex = 2
  val numberReportCardsIndex = 3

  def sequenceOp(reportCardMetrics : Array[Int],
    reportCardContent : String) : Array[Int] = {

    val words = reportCardContent.split(" ")
    //modify each of the elements in the array
    reportCardMetrics(totalWordIndex) += words.length
    reportCardMetrics(longestWordIndex) = Math.max(
      reportCardMetrics(longestWordIndex),
      words.map(w => w.length).max)
    reportCardMetrics(happyMentionsIndex) += words.count(
      w => w.toLowerCase.equals("happy"))
    reportCardMetrics(numberReportCardsIndex) +=1
    reportCardMetrics
  }

  def compOp(x : Array[Int], y : Array[Int]) : Array[Int] = {
    //combine the first and second arrays by modifying the
    elements
    // in the first array
    x(totalWordIndex) += y(totalWordIndex)
    x(longestWordIndex) = Math.max(x(longestWordIndex),
y(longestWordIndex))
    x(happyMentionsIndex) += y(happyMentionsIndex)
    x(numberReportCardsIndex) += y(numberReportCardsIndex)
    x
  }

  def toReportCardMetrics(ar : Array[Int]) : ReportCardMetrics =
    ReportCardMetrics(
      ar(longestWordIndex),
      ar(happyMentionsIndex),
      ar(totalWordIndex)/ar(numberReportCardsIndex)
    )
}
```

We would then need to modify our aggregation code slightly. We are not using the same custom aggregation object, and the zero value has changed.

This is shown in [Example 5-7](#).

Example 5-7. Aggregation with arrays to minimize expensive object creation

```
def calculateReportCardStatisticsWithArrays(rdd : RDD[(String,
String)])
  : RDD[(String, ReportCardMetrics)] = {

  rdd.aggregateByKey(
    //the zero value is a four element array of zeros
    Array.fill[Int](4)(0)
  )(
    //seqOp adds the relevant values to the array
    seqOp = (reportCardMetrics, reportCardText) =>
      MetricsCalculator_Arrays.sequenceOp(reportCardMetrics,
reportCardText),
    //combo defines how the arrays should be combined
    combOp = (x, y) => MetricsCalculator_Arrays.compOp(x, y))
    .mapValues(MetricsCalculator_Arrays.toReportCardMetrics)
}
```

Within a function, it is often beneficial to avoid intermediate object creation. It is important to remember that converting between types (such as between different flavors of Scala collections) creates intermediate objects. This is yet another place in which implicit conversions may have unfortunate performance implications.

For example, suppose that observing our note in the previous section, you wanted to speed up the sequence function of the `MetricsCalculator_ReuseObjects` object. Then, you realized that your coworker had written a general-purpose utility that finds the instances of the word “happy” and the longest word in a collection of strings (shown in [Example 5-8](#)).

Example 5-8. Function with implicit sequence conversions

```
def findWordMetrics[T <: Seq[String]](collection : T ) : (Int,
Int)={
  val iterator = collection.toIterator
```

```

var mentionsOfHappy = 0
var longestWordSoFar = 0
while(iterator.hasNext){
  val n = iterator.next()
  if(n.toLowerCase == "happy"){
    mentionsOfHappy +=1
  }
  val length = n.length
  if(length> longestWordSoFar) {
    longestWordSoFar = length
  }
}
(longestWordSoFar, mentionsOfHappy)

```

Your coworker helpfully defined her function on any type that extends a Scala `Traversable` index. Thus, you won't need to convert the array of words at all and can happily write the code shown in [Example 5-9](#).

Example 5-9. Aggregation with bad implicit conversions

```

val totalWordIndex = 0
val longestWordIndex = 1
val happyMentionsIndex = 2
val numberReportCardsIndex = 3
def fasterSeqOp(reportCardMetrics : Array[Int], content :
String): Array[Int] = {
  val words: Seq[String] = content.split(" ")
  val (longestWord, happyMentions) =
CollectionRoutines.findWordMetrics(words)
  reportCardMetrics(totalWordIndex) += words.length
  reportCardMetrics(longestWordIndex) = longestWord
  reportCardMetrics(happyMentionsIndex) += happyMentions
  reportCardMetrics(numberReportCardsIndex) +=1
  reportCardMetrics
}

```

Unfortunately, in terms of object creation, this new implementation is actually worse than the previous one. It creates two extra objects containing the collection with the words each time a sequence operation is called! First when you call the `findWordMetrics` routine, since the input array has to be implicitly converted to a `Traversable` object

(creating a new object of the same size), and again when your coworker's code casts the `Traversable` object to an `Iterator`.

WARNING

Modifying a value passed into your transformation is not always safe, so double-check the documentation for the function you are using.

TIP

Beyond reducing the objects that are directly allocated, Scala's implicit conversions can sometimes cause additional allocations in the process of converting.

Iterator-to-Iterator Transformations with `mapPartitions`

The RDD `mapPartitions` function takes as its argument a function from an `iterator` of records (representing the records on one partition) to another `iterator` of records (representing the output partition).

The `mapPartitions` transformation is one of the most powerful in Spark since it lets the user define an arbitrary routine on one partition of data. The `mapPartitions` transformation can be used for very simple data transformations like string parsing, but it can also be used for complex, expensive data-processing work to solve problems such as secondary sort or highly custom aggregations. Many of Spark's other transformations, like `filter`, `map`, and `flatMap`, can be built using `mapPartitions`. Optimizing the `mapPartitions` routines is an important part of writing complicated and performant Spark code, as we

will see in [Chapter 6](#). To allow Spark the flexibility to spill some records to disk, it is important to represent your functions inside of `mapPartitions` in such a way that your functions do not force loading the entire partition in-memory (e.g., implicitly converting to a list). Iterators have many methods we can use to write functional-style transformations. You may also construct your own custom iterator extending the `Iterator` interface. When a transformation directly takes and returns an iterator without forcing it through another collection, we call it an *iterator-to-iterator* transformation.

What Is an Iterator-to-Iterator Transformation?

A Scala `iterator` object is not actually a collection, but a function that defines a process of accessing the elements in a collection one-by-one. Not only are iterators immutable, but the same element in an iterator can only be accessed one time. In other words, iterators can only be traversed once, and they extend the Scala interface `TraversableOnce`. Iterators have some of the same methods defined on them as other immutable Scala collections, such as mappings (`map` and `flatMap`), additions (`++`), folds (`foldLeft`, `reduceRight`, `reduce`), element conditions (`forall` and `exists`), and traversals (`next` and `foreach`). In some instances, these methods behave differently than other Scala collections. Since the iterator can only be traversed once, any of the iterator methods that require looking at all the elements in the iterator will leave the original iterator empty.

NOTE

Java has its own implementation of iterators, `java.util.Iterator`, which have the same benefits as Scala iterators for Spark's evaluation.

WARNING

Beware of your function calls. It is easy to accidentally consume an iterator by calling an object that traverses through the iterator such as `size`, or to trigger an implicit conversion. Iterators can be converted to any other Scala collection type. However, converting them requires accessing each of the elements. Thus, after it has been converted to a new collection type, an iterator will be at its last element (empty).

In some ways it can be helpful to conceptualize iterator methods as we would RDD methods—as either *transformations* or *actions*—because like an RDD, an iterator is actually a set of evaluation instructions rather than a stored state. Some iterator methods, like `next`, `size`, and `foreach`, traverse the iterator and evaluate it (more like an action). Others, like `map` and `flatMap`, return a new iterator—which is really a set of evaluation instructions—much like RDD transformations return a new RDD. However, in contrast to Spark transformations, iterator transformations are executed linearly, one element at a time, rather than in parallel. This makes iterators slower but much easier to use than if they could be executed in parallel. For example, if we needed to store some information about the records we have seen, we can do that in a `filter` or a `map` function on the iterator, since the map/filter routine will be applied to each element sequentially. (See [Example 5-12](#) at the end of this section.) One-to-one functions are also not chained together in iterator operations so using three `map` calls still requires looking at each element in the iterator three times.

By “iterator-to-iterator transformation” we mean using one of these iterator “transformations” to return a new iterator rather than a) converting the iterator to a different collection or b) evaluating the iterator with one of